MVLU COLLEGE
WSN PRACTICAL

Aim:Simulation of Single Mote using TOSSIM
coide:import heapq
print("hariprasad vishwakarma")

```
class MockNode:
    """Stands in for a single TOSSIM node (e.g. node 0 running BlinkC)."""

    def __init__(self, node_id):  # Fixed typo: changed _init_ to __init__
        self.node_id = node_id
        self.boot_time = None

    def boot_at_time(self, time):
        self.boot_time = time

class MockTossim:
    """
    Stands in for the Tossim object. Holds a priority queue ("event
    queue") of (time, description) pairs -- this mirrors how real
    TOSSIM jumps directly from event to event rather than advancing
    time continuously.
    """

    TIMER_PERIOD = 512 # ticks between timer fires, like Blink's default

    def __init__(self):  # Fixed typo: changed _init_ to __init__
        self.event_queue = [] # heap of (time, event_description)
        self.nodes = {}

    def get_node(self, node_id):
        node = MockNode(node_id)
        self.nodes[node_id] = node
        return node

    def schedule_boot(self, node):
        heapq.heappush(
            self.event_queue,
            (node.boot_time, f"Booting node {node.node_id} at time {node.boot_time}.")
        )
        # Real TinyOS starts the app's timer right after boot; we mimic that
        # by scheduling the first timer fire one period after boot.
        first_fire = node.boot_time + self.TIMER_PERIOD
        heapq.heappush(
            self.event_queue,
            (first_fire, f"BlinkC: Timer 0 fired @ {first_fire}.")
```

T127
Hariprasad Vishwakarma
TYCS

Hariprasad

```
        )

    def run_next_event(self):
        """Pop and print the earliest event, then schedule the next timer fire."""
        if not self.event_queue:
            return False

        time, description = heapq.heappop(self.event_queue)
        print(f"DEBUG ({self._active_node_id()}): {description}")

        # If this was a timer fire, schedule the next one -- this is how
        # a periodic timer keeps producing events indefinitely.
        if "Timer 0 fired" in description:
            next_fire = time + self.TIMER_PERIOD
            heapq.heappush(
                self.event_queue,
                (next_fire, f"BlinkC: Timer 0 fired @ {next_fire}.")
            )
        return True

    def _active_node_id(self):
        # Safety check: if nodes dict is empty, return 0
        if not self.nodes:
            return 0
        return next(iter(self.nodes))

# ---- Simulation setup, mirrors the real sim.py structure ----

t = MockTossim()

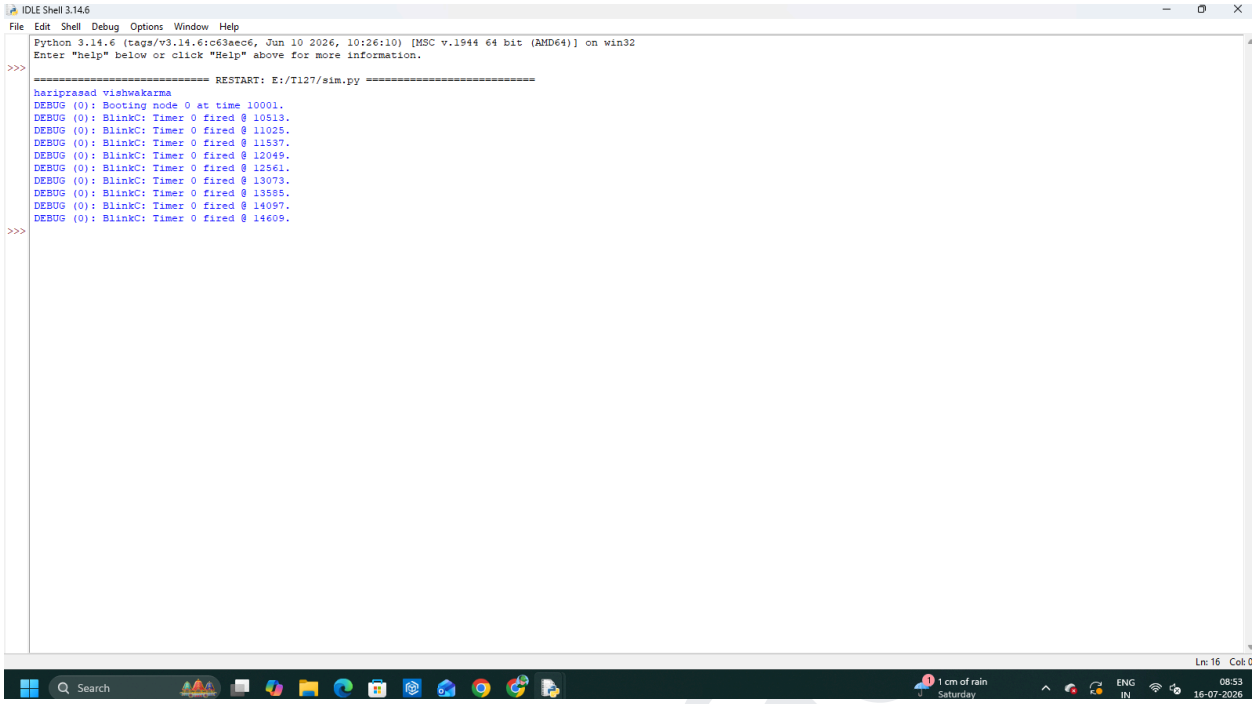
# Create node 0
node = t.get_node(0)

# Set the boot time (matches the practical's example)
node.boot_at_time(10001)
t.schedule_boot(node)

# Run the simulation for a fixed number of events
for i in range(10):
    t.run_next_event()
```

Hariprasad

output:



Hariprasad